

API Data Flow Reference

System overview

PetPals uses a two-token authentication system. Authentication credentials are centralised in a single USERS table shared across all six role-specific tables (Admin, Staff, Veterinarian, Adopter, Volunteer, Donor).

Token	Storage	Expiry	Purpose
Access token	Zustand memory (frontend)	15 minutes	Sent in Authorization header on every protected API request.
Refresh token	httpOnly cookie (browser)	7 days	Used only to issue new access tokens - never sent in API headers

Architecture pattern followed by every backend endpoint:

Route → Middleware (protected routes only) → Controller → Service → Database

Frontend auth pattern on every page load:

App.tsx mounts → refreshToken() called → httpOnly cookie sent → new access token returned → Zustand populated

1. Backend Auth API

Base path: /api/v1/auth

Four endpoints covering registration, login, logout, and token refresh. Register and login are public. Logout requires a valid Bearer access token. Refresh-token requires only the httpOnly cookie.

1.1 POST /api/v1/auth/register

Creates a new user account. Writes two rows atomically - one to USERS (credentials) and one to the role-specific table (name + userID). The Prisma \$transaction ensures both succeed or both roll back.

Request summary

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	name, email, password, role
Valid roles	admin, adopter, staff, vet, volunteer, donor
Success	201 - { userID, userEmail, role }

Data flow

File / Layer	What happens here
auth.routes.js	Receives POST request, no middleware - public route. Maps POST /register → register controller
auth.controller.js	Validates all 4 fields are present and role is one of the 6 valid values. Deconstructs { name, email, password, role } from req.body. Returns VALIDATION_ERROR (422) if any field is missing. Returns VALIDATION_ERROR (422) if role is not valid. Calls authService.register() in try/catch. On success: successResponse 201 with created user. On failure: next(err) → errorHandler
auth.service.js	Duplicate email check, bcrypt hash, atomic double-insert via Prisma transaction. findUnique on USERS by email - throws CONFLICT (409) if exists. bcrypt.hash(password, 10). Opens

	\$transaction: creates USERS row, then role-specific row using returned userID. If either insert fails, entire transaction rolls back - no orphaned records. Strips userPassword from returned object
USERS table	Stores auth credentials. Columns written: userEmail, userPassword (hashed), role
Role table	Stores name and userID for the specific role. Columns written: userID (FK → USERS), [role]Name (e.g. adopterName)

accountStatus at registration: accountStatus is set explicitly for four of the six roles - Admin, Staff, Veterinarian, and Volunteer - which start as "Pending" and require approval (via the PATCH .../status endpoints) before they can operate. The very first Admin registered in the system auto-activates as "Active" immediately, since there is no existing admin to approve them. Adopter and Donor registrations start "Active" by their own table defaults. Staff, Vet, and Volunteer choose a shelter at sign-up (shelterID is required), and the shelter must be Open (422 otherwise). A vet can register only at a shelter that has a manager (422 "This shelter isn't onboarding veterinarians yet - it has no manager assigned"). The first Staff sign-up at a shelter with no manager (and no pending Manager sign-up) registers as that shelter's Manager and is approved by an Admin; other staff and vets are approved by that shelter's Manager, and volunteers by any staff member at their shelter. Staff, Veterinarians and Volunteers onboard BEFORE approval: after registering they complete an onboarding wizard (personal details, address, government ID) - see 1.6. Staff and Veterinarians are signed straight in by the worker-portal sign-up page; Volunteers are not workers and use the public /login page. Admin accounts wait for approval before they can sign in. Adopter and Donor accounts are already Active and have a skippable onboarding wizard (see 1.7).

Error cases

Scenario	Error Code	Response Message
Missing any of the 4 fields	VALIDATION_ERROR	name, email, password, and role are required
Role not in the allowed list	VALIDATION_ERROR	role must be one of: admin, adopter, staff, vet, volunteer, donor
Shelter is not Open (Staff or Vet sign-up)	VALIDATION_ERROR	Shelter is not Open
Vet signs up at a shelter with no manager	VALIDATION_ERROR	This shelter isn't onboarding veterinarians yet - it has no manager assigned
Email already registered	CONFLICT	A user with this email is already registered
DB insert fails	INTERNAL_SERVER_ERROR	Internal server error

1.2 POST /api/v1/auth/login

Authenticates an existing user. Verifies credentials against USERS, issues a 15m access token in the response body and a 7d refresh token in an httpOnly cookie. Also fetches the user's name from the role-specific table for UI display.

Request summary

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	email, password
Success	200 - { token (15m), user: { userID, userEmail, role, name, avatarSeed, onboardingComplete, onboardingStep (Adopter/Staff/Veterinarian/Volunteer/Donor), accountStatus (Staff/Veterinarian/Volunteer) } } + refreshToken httpOnly cookie (7d)

Data flow

File / Layer	What happens here
--------------	-------------------

auth.routes.js	Receives POST request, no middleware – public route. Maps POST /login → login controller
auth.controller.js	Validates fields, calls service, signs both tokens, sets cookie. Returns <code>VALIDATION_ERROR</code> (422) if email or password missing. Calls <code>authService.login()</code> in try/catch. Signs access token payload: { <code>userID</code> , <code>role</code> } for all roles, 15m expiry, <code>JWT_SECRET</code> . Signs refresh token: { <code>userID</code> }, 7d expiry, <code>JWT_SECRET</code> . Hashes refresh token with <code>bcrypt</code> and calls <code>authService.storeRefreshToken(userID, hashedRT)</code> . Sets <code>refreshToken</code> as <code>httpOnly</code> cookie (<code>sameSite: strict</code> , <code>secure</code> in production). Returns 200 with { <code>token: accessToken, user</code> }
auth.service.js (login)	Verifies credentials, fetches name from role table, returns safe user object. <code>findUnique</code> on <code>USERS</code> by email (selects <code>userID</code> , <code>userEmail</code> , <code>role</code> , <code>userPassword</code> only). Throws <code>NOT_FOUND</code> (404) if no record. <code>bcrypt.compare(password, user.userPassword)</code> - throws <code>UNAUTHORIZED</code> (401) if no match. Queries role-specific table for the user's name using <code>ROLE_CONFIG</code> . Checks the account status: <code>Deactivated</code> and <code>Banned</code> accounts are refused for every role, and <code>Pending</code> accounts are refused unless the role is in <code>PENDING_LOGIN_ROLES</code> (<code>Staff</code> , <code>Veterinarian</code> , <code>Volunteer</code>). Strips <code>userPassword</code> before returning { <code>...safeUser</code> , <code>name</code> }
auth.service.js (storeRefreshToken)	Stores hashed refresh token in <code>USERS</code> table. <code>prisma.users.update({ where: { userID }, data: { refreshToken: hashedRT } })</code>
<code>USERS</code> table	Queried for credentials, updated with refresh token hash. Columns read: <code>userEmail</code> , <code>userPassword</code> . Columns written: <code>refreshToken</code> (hashed)
Role table	Queried for the user's display name. e.g. <code>Adopter.adopterName</code> , <code>Staff.staffName</code> etc. <code>avatarSeed</code> is also selected here for every role (not <code>Staff</code> -specific) and included in the returned user object. For <code>Adopter</code> , <code>Staff</code> , <code>Veterinarian</code> , <code>Volunteer</code> and <code>Donor</code> the onboarding progress (<code>onboardingComplete</code> , <code>onboardingStep</code>) is selected too, and <code>accountStatus</code> for <code>Staff</code> , <code>Veterinarian</code> and <code>Volunteer</code> .

Design note — `shelterID` is not in the token: the payload is { `userID`, `role` } for every role. `Staff`-scoped endpoints re-fetch the staff member's current `shelterID` from `STAFF` on each request, so a shelter reassignment takes effect immediately.

`lastLoginAt` lives on `USERS` and is updated on every successful login, for all roles.

Error cases

Scenario	Error Code	Response Message
Missing email or password	<code>VALIDATION_ERROR</code>	email and password are required
Email not found in <code>USERS</code>	<code>NOT_FOUND</code>	No account found with this email
Password does not match hash	<code>UNAUTHORIZED</code>	Incorrect password
Account status is <code>Pending</code> (<code>Admin</code>)	<code>UNAUTHORIZED</code>	This account is pending approval and can't log in yet.
Account status is <code>Deactivated</code> or <code>Banned</code> (every role)	<code>UNAUTHORIZED</code>	This account is no longer active

Pending `Staff`, Pending `Veterinarian` and Pending `Volunteer` accounts can log in: login succeeds so they can onboard before approval, but `authenticate` admits them only on their own onboarding routes (see 1.5). The roles whose `Pending` accounts can log in are defined once as `PENDING_LOGIN_ROLES = { Staff, Veterinarian, Volunteer }` in `server/src/services/auth/pendingRoles.js`, shared by `login()` in `auth.service.js` and the `authenticate` middleware. `Pending Admin` accounts cannot log in (401), and `Deactivated` and `Banned` accounts are refused for every role. Every login refusal is a 401, never a 403: the frontend's `axios` interceptor redirects every 403 to `/forbidden`, which would hide the message on the login page. The success response carries `onboardingComplete` and `onboardingStep` for `Adopter`, `Staff`, `Veterinarian`, `Volunteer` and `Donor`, and `accountStatus` for `Staff`, `Veterinarian` and `Volunteer`. A `Donor`, like an `Adopter`, carries no `accountStatus`: `Donors` are `Active` from sign-up.

1.3 POST /api/v1/auth/logout

Ends the session by nullifying the stored refresh token in the DB and clearing the httpOnly cookie. The access token is not denylisted - with a 15-minute expiry the risk window is acceptable. The refresh token nullification is the source of truth for session invalidation.

Request summary

Field	Value
Method	POST
Auth required	Yes — Bearer access token (a Pending Staff member, Veterinarian or Volunteer may log out too)
Body fields	None
Success	200 - { message: "Logged out successfully" }

Data flow

File / Layer	What happens here
auth.routes.js	Registers authenticate middleware before controller. Maps POST /logout to: authenticate.allowPending → logout controller
authenticate.js	Validates the Bearer access token. Extracts token from Authorization header via optional chaining. Returns UNAUTHORIZED if missing. jwt.verify(token, JWT_SECRET) - throws if invalid or expired. Attaches { userID, role } to req.user, checks the account status live (see 1.5), and calls next()
auth.controller.js	Calls service with userID and clears the cookie. Uses req.user.userID (set by authenticate middleware). Calls authService.logout(userID) in try/catch. res.clearCookie('refreshToken') - removes httpOnly cookie from browser. Returns 200 with 'Logged out successfully'
auth.service.js	Nullifies stored refresh token in DB. prisma.users.update({ where: { userID }, data: { refreshToken: null } }). Any future refresh attempt will fail bcrypt comparison
USERS table	refreshToken column set to null. Columns written: refreshToken = null

Error cases

Scenario	Error Code	Response Message
No Authorization header	UNAUTHORIZED	Authentication failed - no token provided
Access token invalid or expired	UNAUTHORIZED	Authentication failed - invalid token

1.4 POST /api/v1/auth/refresh-token

Issues a new access token and rotates the refresh token. Does not require an access token - authentication is via the httpOnly cookie only. Called by the frontend on every app load to restore session from the cookie.

Request summary

Field	Value
Method	POST
Auth required	No - cookie only (no Bearer token needed)
Cookie required	refreshToken httpOnly cookie
Body fields	None
Success	200 - { token (new access token, 15m), user: { userID, role, name, avatarSeed, onboardingComplete, onboardingStep (Adopter/Staff/Veterinarian/Volunteer/Donor), accountStatus (Staff/Veterinarian/Volunteer) } } + new refreshToken cookie (7d)

Data flow

File / Layer	What happens here
auth.routes.js	No middleware - cookie-only authentication handled in controller. Maps POST

	/refresh-token → refreshToken controller directly
auth.controller.js	Reads cookie, decodes userID from it, signs new tokens, calls service. Reads rawOldRT from req.cookies.refreshToken - returns UNAUTHORIZED if missing. jwt.decode(rawOldRT) to extract userID (no signature verification here). Returns UNAUTHORIZED if decoded.userID is absent. Signs new refresh token: { userID }, 7d expiry. Calls authService.refreshToken(userID, rawOldRT, newRefreshToken) in try/catch. On success: signs new access token payload { userID, role }, 15m expiry. Sets new refreshToken as httpOnly cookie. Returns 200 with new access token and { userID, role, name, avatarSeed, onboardingComplete, onboardingStep, accountStatus } (the onboarding fields for Adopter, Staff, Veterinarian, Volunteer and Donor; accountStatus for Staff, Veterinarian and Volunteer). Design note: this response object is built from an explicit field whitelist in the controller ({ userID, role, name, avatarSeed, onboardingComplete, onboardingStep, accountStatus }). Any field added to the service's return value must also be added to this whitelist explicitly, or it is dropped without error.
auth.service.js	Verifies old refresh token against DB hash, stores new hash, fetches name and avatarSeed. findUnique on USERS by userID (selects userID, role, refreshToken). Throws NOT_FOUND if user missing. Throws UNAUTHORIZED if refreshToken in DB is null (already logged out). bcrypt.compare(rawOldRT, user.refreshToken) - throws UNAUTHORIZED if no match. bcrypt.hash(rawNewRT, 10) and updates USERS.refreshToken with new hash. Queries role-specific table for the user's name and avatarSeed using ROLE_CONFIG. Returns { userID, role, name, avatarSeed } (refreshToken stripped).
USERS table	refreshToken read for verification, updated with new hash. Columns read: refreshToken (hash for bcrypt comparison), role. Columns written: refreshToken (new hash)

Error cases

Scenario	Error Code	Response Message
No refreshToken cookie	UNAUTHORIZED	No refresh token provided
Cookie cannot be decoded	UNAUTHORIZED	Invalid refresh token
User not found in DB	NOT_FOUND	User not found
refreshToken null in DB (logged out)	UNAUTHORIZED	Session expired, please log in again
Cookie does not match DB hash	UNAUTHORIZED	Invalid refresh token

Design note - rotation

Each refresh token can only be used once. After a successful refresh the old hash is replaced in the DB. If an attacker steals a refresh token and tries to use it after the legitimate user has already rotated it, bcrypt.compare will fail and the request is rejected.

1.5 Live Account-Status Enforcement

Every protected request verifies the Bearer token and then checks the account's status live, so an access token never outlives the account's standing. middleware/authenticate.js calls the auth.service.js export getAccountStatus(userID, role) on every protected request (one indexed lookup on the role table's primary key), not only at login and refresh. If the account's current status is Deactivated, Banned, Pending, or the sentinel “DELETED” (returned when the role's row no longer exists at all - e.g. after a completed self-delete), the request is rejected with 401 UNAUTHORIZED, message “This account is no longer active”, even if the JWT signature and expiry are otherwise valid. This applies to every role and every protected route: an admin banning an adopter, a manager approving, declining or deactivating a staff member or vet, or an account completing a self-delete takes effect on the next request, without re-login.

The live status check is a DB read in addition to the JWT verification step, not a replacement for it - signature and expiry checks happen first.

The middleware is built by one factory, buildAuthenticate({ allowPending }), which produces two exports: authenticate (allowPending false) and authenticate.allowPending (allowPending true). On routes wired with

authenticate.allowPending, a Pending account whose role is in PENDING_LOGIN_ROLES is admitted; any other Pending role is still blocked. PENDING_LOGIN_ROLES = { Staff, Veterinarian, Volunteer } is defined once in server/src/services/auth/pendingRoles.js and is shared by login() and the middleware. Every other route uses plain authenticate, so a Pending staff member, vet or volunteer receives 401 everywhere else.

allowPending routes

Role	allowPending routes
Staff	GET/PUT /staff/me, GET/POST /staff/me/government-id, PATCH /staff/me/onboarding-step, PATCH /staff/me/onboarding-complete
Veterinarian	GET/PUT /vets/me, GET/POST /vets/me/government-id, PATCH /vets/me/onboarding-step, PATCH /vets/me/onboarding-complete
Volunteer	GET/PUT /volunteers/me, GET/POST /volunteers/me/government-id, PATCH /volunteers/me/onboarding-step, PATCH /volunteers/me/onboarding-complete
All three	POST /auth/logout

The allowPending route list is deliberately minimal and has the same shape for all three roles. Approving a Pending account requires completed onboarding and a Verified government ID (see 1.6).

Volunteers: every other volunteer route rejects a Pending volunteer with 401, including PUT /volunteers/me/availability, GET /volunteers/me/tasks, /volunteers/me/events, /volunteers/me/appointments and DELETE /volunteers/me. The /volunteers/me routes are mounted before the staff /volunteers/:id router, so that /volunteers/me is not captured by /volunteers/:id.

1.6 Staff, Vet and Volunteer Sign-up, Onboarding and Approval

Staff, vets and volunteers register with a shelterID. The shelter must be Open (422 otherwise). A vet can register only at a shelter that has a manager: 422 "This shelter isn't onboarding veterinarians yet - it has no manager assigned".

Onboarding wizard

New staff, vets and volunteers start Pending and complete a mandatory onboarding wizard before approval. Completing onboarding (PATCH .../onboarding-complete) returns 409 naming anything that is missing.

Step	Screen
2	Personal
3	Address
4	Identity (government ID upload)
5	Review

Approval

Account	Approved by	Endpoint
Staff	Shelter Manager	PATCH /staff/me/team/:id/status
Veterinarian	Shelter Manager	PATCH /staff/me/vets/:id/status
Volunteer	Any staff member at the volunteer's shelter	PATCH /volunteers/:id/status
Manager sign-up	Admin	PATCH /staff/:id/status

Approval requires completed onboarding AND a Verified government ID; otherwise the request returns 409 naming what is missing. The rule is enforced in services/staff/staffApproval.service.js, which every approval path shares (keyed by userType Staff, Veterinarian or Volunteer). Declining is allowed at any stage. A volunteer may be approved by any staff member at the volunteer's shelter, not only the Manager. Because account status is checked on every request (see 1.5), approval or deactivation takes effect on the next request without re-login.

Pre-existing approved accounts are marked as onboarded by the data migrations `staff_onboarding_backfill` and `vet_onboarding_backfill`, so the wizard applies only to new sign-ups. Existing non-Pending volunteers were likewise backfilled as onboarded.

1.7 Donor Sign-up and Skippable Onboarding

Donors register with `POST /auth/register` (role "donor") and no shelter. A donor account is Active from sign-up: there is no approval and no government ID, so a donor is never Pending and has no `allowPending` routes. Login and refresh-token responses carry `onboardingComplete` and `onboardingStep` but not `accountStatus`, the same as an Adopter.

Onboarding wizard

Step	Screen
2	Personal
3	Address
4	Review

Unlike the approval roles, the donor wizard is skippable. "Skip for now" lets the donor use the dashboard and donate for the current browser session, and the next login brings them back to the wizard (see 4.1, `OnboardingGate`). Donating never waits on onboarding.

2. Backend Supporting Files

Shared across all auth endpoints

File	Purpose
<code>config/prisma.js</code>	Singleton Prisma client using <code>PrismaPg</code> adapter. Single gateway to the DB - imported by all service files.
<code>utils/response.js</code>	<code>successResponse()</code> and <code>errorResponse()</code> - standardise all API response shapes.
<code>utils/errors.js</code>	<code>ERROR_CODES</code> map - defines valid error codes and their HTTP status codes (400, 401, 403, 404, 409, 422, 500).
<code>middleware/errorHandler.js</code>	Global error handler registered last in <code>app.js</code> . Receives errors via <code>next(err)</code> , maps <code>err.code</code> to HTTP status, formats using <code>errorResponse()</code> .
<code>middleware/authenticate.js</code>	JWT verification and live account-status middleware, built by the <code>buildAuthenticate({ allowPending })</code> factory. Extracts Bearer token, verifies signature, attaches <code>{ userID, role }</code> to <code>req.user</code> - then calls <code>authService.getAccountStatus(userID, role)</code> and rejects with 401 if the account is Deactivated, Banned, Pending, or DELETED (see 1.5). Exports <code>authenticate</code> (<code>allowPending</code> false) and <code>authenticate.allowPending</code> (<code>allowPending</code> true: also admits Pending accounts whose role is in <code>PENDING_LOGIN_ROLES</code>). Used on <code>logout</code> (<code>allowPending</code> variant). Not used on refresh-token.
<code>middleware/authorizeRoles.js</code>	RBAC middleware. Takes <code>...allowedRoles</code> , checks <code>req.user.role</code> against the list, returns 403 FORBIDDEN if not permitted. Applied to every role-specific endpoint.
<code>services/auth/pendingRoles.js</code>	Defines <code>PENDING_LOGIN_ROLES = { Staff, Veterinarian, Volunteer }</code> once - the roles whose Pending accounts can log in and reach their <code>allowPending</code> routes. Shared by <code>login()</code> in <code>auth.service.js</code> and the <code>authenticate</code> middleware.
<code>services/staff/staffApproval.service.js</code>	Single approval rule shared by every staff, vet and volunteer approval path (keyed by <code>userType</code> <code>Staff</code> , <code>Veterinarian</code> or <code>Volunteer</code>): <code>onboarding complete AND government ID Verified</code> , otherwise 409 naming what is missing. Declining is always allowed.
<code>app.js</code>	Express app setup - middleware, routes, error handler. No <code>app.listen()</code> call. Imported by tests directly.
<code>index.js</code>	Thin entry point - requires <code>app.js</code> and calls <code>app.listen()</code> . Split from <code>app.js</code> to allow Supertest to import the app without binding a port.

3. Backend Supporting Files — client/src | Sprint 1

The frontend auth system is built around four files that each have a single responsibility. They are intentionally decoupled - the API layer knows nothing about state, the state layer knows nothing about HTTP, and the UI layer composes both.

3.1 File responsibilities

File	Responsibility
api/authApi.ts	API functions for auth endpoints - register(), login(), refreshToken(), logout(). Each makes an axios call and returns typed data. Knows nothing about state.
api/axiosInstance.ts	Pre-configured axios client. Sets baseUrl from VITE_API_URL env variable, withCredentials: true (sends httpOnly cookie automatically). Request interceptor attaches Bearer token from Zustand. Response interceptor handles 401 (clear state + redirect to the login page for the role: /staff-portal/login for Admin/Staff/Veterinarian, /login otherwise) and 403 (redirect to /forbidden), skipping auth endpoints to avoid redirect loops.
store/useAuthStore.ts	Zustand global store. Holds { user, token, role } in memory. login() sets all fields, logout() clears them. State is lost on page refresh - restored via refreshToken() on app load.
App.tsx	Root component. On mount, calls refreshToken() to restore session from the httpOnly cookie. On success, populates Zustand. On failure, does nothing - user will see public pages or be redirected to /login by ProtectedRoute. Uses authReady state to prevent flash of unauthenticated content.

3.2 Session restore on page load

Because Zustand state is memory-only, it is lost on every page refresh. The session is restored using the httpOnly cookie which persists across refreshes.

Flow

File / Layer	What happens here
App.tsx	useEffect fires on mount, calls refreshToken(). authReady state starts false - renders a rose background placeholder to prevent flash. refreshToken() called - axiosInstance sends the request with credentials: true. Browser automatically attaches the httpOnly refreshToken cookie
axiosInstance.ts	Sends POST /auth/refresh-token with withCredentials: true. isAuthCall guard on response interceptor prevents redirect loop if this call returns 401
Backend /auth/refresh-token	Verifies cookie, rotates tokens, returns new access token + name. New refreshToken set as httpOnly cookie in response. Returns { token, user: { userID, role, name, avatarSeed, onboardingComplete, onboardingStep, accountStatus } }
App.tsx .then()	On success, populates Zustand and sets authReady = true. storeLogin(user, token, user.role) - stores session in memory. authReady = true - full app renders, Navbar shows user name and dropdown
App.tsx .catch()	On failure (no cookie, expired), silently ignored. authReady = true - app renders normally. User sees public pages; ProtectedRoute redirects to /login if they try to access a dashboard

3.3 Login flow (Login.tsx)

File / Layer	What happens here
Login.tsx	Validates fields client-side, calls authApi.login(), updates Zustand, navigates. emailRegex validation + required field check before hitting API. noValidate on form - browser validation suppressed, custom error box used. On success: if a ?redirect= param is present, go there (unless it points into another role's dashboard); otherwise go to the role's dashboard via resolveDestination (Veterinarian → /vet, Volunteer → /volunteer, Donor → /donor). replace: true prevents back button returning to login page. On failure: displays server error message inline. If user already authenticated (token in Zustand): <Navigate> redirect away

	from login immediately
authApi.ts (login)	POSTs { email, password } to /auth/login. Returns { token, user: { userID, userEmail, role, name, avatarSeed, onboardingComplete, onboardingStep, accountStatus } }. axiosInstance automatically receives and stores the httpOnly refreshToken cookie

3.4 Logout flow (Navbar.tsx)

File / Layer	What happens here
Navbar.tsx (handleLogout)	Calls the API, then leaves and clears the session in ONE render via useEndSession (flushSync around navigate + storeLogout) — clearing the session first would let ProtectedRoute capture the current page as a stale /login?redirect. Destination by role comes from logoutDestinationFor: Admin, Staff and Veterinarian → /staff-portal/login; everyone else (Adopter, Volunteer, Donor) → /.
authApi.ts (logout)	POSTs to /auth/logout with Bearer token from Zustand. axiosInstance request interceptor attaches Authorization: Bearer <token> automatically

3.5 Worker session flow and destinations

Session restore (3.2) rebuilds the session from POST /auth/refresh-token on every app load. The response carries the onboarding fields and accountStatus that the OnboardingGate (4.1) reads to route a Pending staff member, vet or volunteer: unfinished onboarding leads to the onboarding wizard (/staff/onboarding/step/:step, /vet/onboarding/step/:step or /volunteer/onboarding); finished onboarding with approval pending leads to the shared AwaitingApproval page (/staff/pending, /vet/pending or /volunteer/pending); an Active account reaches its dashboard.

Workers (Admin, Staff, Veterinarian) land on the worker login after logging out, after a 401, and after closing their own account. Adopters, Volunteers and Donors go home (/) on log out and to /login on a 401. Volunteers and Donors are not workers: they sign in on the public /login page, closing their own account also ends on /, and after login they land on /volunteer and /donor respectively.

Event	Workers (Admin, Staff, Veterinarian)	Adopters, Volunteers and Donors
Log out	Worker login (/staff-portal/login)	Home (/)
After a 401	Worker login (/staff-portal/login)	/login
After closing their own account	Worker login (/staff-portal/login)	Home (/) (Volunteers, Donors)

The destinations come from three helpers in client/src/logic/route/resolveDestination.ts: logoutDestinationFor, loginPathFor and loginPathForPage. A signed-out visitor who opens /staff, /admin or /vet is sent to the worker login with ?redirect= preserved. A signed-out visitor who opens /volunteer or /donor is sent to /login with ?redirect= preserved.

Ending a session from the UI always goes through useEndSession, which navigates and clears the session inside one flushSync so ProtectedRoute never captures a stale redirect.

3.6 Closing your own account (session side)

DELETE /vets/me, DELETE /staff/me, DELETE /volunteers/me and DELETE /donors/me take { mode: "deactivate" | "delete" } and use plain authenticate, so a Pending account cannot call them (401). deactivate clears the refresh token; delete removes the account's Users row. DELETE /volunteers/me returns 409 with error.details.blockers (["appointments", "tasks"], whichever apply) while the volunteer has upcoming assisted appointments or open tasks; DELETE /donors/me is never blocked. The client then ends the session through useEndSession: Staff and Veterinarians land on the worker login, Volunteers and Donors land on /.

4. Role-Based Access Control — Frontend guards + backend middleware

PetPals enforces RBAC at two layers - the frontend prevents unauthenticated or wrong-role navigation, and the backend enforces the same rules on every API request. Both must be in place: frontend guards are UX, backend enforcement is security.

4.1 Frontend route guards

ProtectedRoute

Wraps any route that requires authentication. Reads token from Zustand - if absent, redirects to `/login?redirect=<path>` — or to `/staff-portal/login?redirect=<path>` for worker dashboards (`/staff`, `/admin`, `/vet`); `/volunteer` and `/donor` are not worker dashboards and use `/login`. If present, renders children.

Usage: `<ProtectedRoute><AdopterDashboard /></ProtectedRoute>`

RoleRoute

Wraps routes that require a specific role. Accepts an `allowedRoles` prop. Reads role from Zustand - if role is not in the allowed list, redirects to `/forbidden`. Used inside `ProtectedRoute`.

Usage: `<ProtectedRoute><RoleRoute allowedRoles={['Adopter']}><AdopterDashboard /></RoleRoute></ProtectedRoute>`

OnboardingGate

Wraps every route and holds a signed-in user in their onboarding until it is done. There are two kinds of role. Approval roles (Staff, Veterinarian, Volunteer) are held in their wizard until onboarded, then on the shared `AwaitingApproval` page until approved; an Active account reaches its dashboard. Skippable roles (Adopter, Donor) are sent to their wizard while onboarding is unfinished, but may choose "Skip for now".

"Skip for now" sets a `sessionStorage` flag that lifts the gate for the current browser session only. For an Adopter, `/adopt/apply` stays gated even after skipping; a Donor has no gated route, so a skipped Donor can use everything, including donating. The flag is cleared on the next real login, which brings the user back to the wizard.

Role	Type	Wizard	Awaiting approval	Skip for now
Staff	Approval	<code>/staff/onboarding</code>	<code>/staff/pending</code>	No
Veterinarian	Approval	<code>/vet/onboarding</code>	<code>/vet/pending</code>	No
Volunteer	Approval	<code>/volunteer/onboarding</code>	<code>/volunteer/pending</code>	No
Adopter	Skippable	<code>/onboarding</code>	-	Yes (<code>/adopt/apply</code> stays gated)
Donor	Skippable	<code>/donor/onboarding</code>	-	Yes (no gated route)

Route configuration in App.tsx

Route	ProtectedRoute	RoleRoute allowedRoles
<code>/adopter</code>	Yes	Adopter
<code>/staff</code>	Yes	Staff
<code>/vet</code>	Yes	Veterinarian
<code>/volunteer</code>	Yes	Volunteer
<code>/donor</code>	Yes	Donor
<code>/admin</code>	Yes	Admin
<code>/login</code>	No (redirects if already logged in)	-
<code>/register</code>	No (redirects if already logged in)	-
<code>/forbidden</code>	No	-

4.2 Backend RBAC middleware

authorizeRoles.js

Factory middleware that accepts a list of allowed roles and returns an Express middleware function. Called after authenticate so req.user.role is already set.

```
Usage: router.post('/pets', authenticate, authorizeRoles('Staff', 'Admin'),
        createPet)
```

Applied to every protected resource endpoint.

ROLES constant

Key	Value (DB enum)
ROLES.ADMIN	Admin
ROLES.STAFF	Staff
ROLES.ADOPTER	Adopter
ROLES.VETERINARIAN	Veterinarian
ROLES.VOLUNTEER	Volunteer
ROLES.DONOR	Donor

4.3 Forbidden page

A simple public page at /forbidden rendered when RoleRoute or the axiosInstance 403 handler redirects an authenticated user who lacks the required role. Uses the same background image and card style as the login page.

Triggered by: RoleRoute (wrong role on frontend route) and axiosInstance response interceptor (403 from any API call).

5. Auth System Tests — Jest + Supertest | Sprint 1

Test file	Type	Cases covered
tests/unit/auth/authenticate.test.js	Unit	No token → 401, Invalid JWT → 401, Expired JWT → 401, Valid JWT → sets req.user and calls next(), allowPending admits only Pending Staff, Veterinarian and Volunteer accounts (any other Pending role stays blocked)
tests/unit/authorizeRoles.test.js	Unit	Role in allowedRoles → next(), Role NOT in allowedRoles → 403, Multiple allowed roles each pass
tests/unit/auth/login.test.js	Unit	Pending Staff/Vet/Volunteer login and refresh → 200 with accountStatus and onboarding progress; Deactivated → 401; Pending Admin → 401 with no session cookie
tests/unit/vet/vets.selfService.test.js	Unit	Pending vets reach only their onboarding routes (GET/PUT /vets/me, government ID, onboarding-step, onboarding-complete); every other route → 401
tests/unit/staff/staff.selfService.test.js	Unit	Pending staff reach only their onboarding routes (GET/PUT /staff/me, government ID, onboarding-step, onboarding-complete); every other route → 401
tests/integration/auth.register.test.js	Integration	Valid payload → 201 + DB rows created, Missing fields → 422, Duplicate email → 409, Password hashed in DB
tests/integration/auth.login.test.js	Integration	Valid credentials → 200 + JWT, Wrong password → 401, Unregistered email → 404, JWT contains correct userID and role
tests/integration/	Integration	Valid token → 200 + DB refreshToken null, Stale refresh token

auth.logout.test.js		after logout → 401, No token → 401
tests/integration/ auth.refreshToken.test.js	Integration	Valid cookie → 200 + new JWT
tests/integration/staff/ staffOnboarding.journey.test.js	Integration	Sign up → Pending login → onboard → ID verified → manager approves → full access
tests/integration/vet/ sprint6Journeys.test.js	Integration	Register → Pending login → onboarding → government ID → manager verifies ID → manager approves → full access

All integration tests seed data via the real register/login endpoints and clean up via Prisma in afterAll. Tests run sequentially (--runInBand) to avoid FK conflicts on the shared Supabase dev DB.